



1	Project overview	8	API layer
2	Folder structure	9	Models
3	File map	10	Navigation
4	Entry point	11	Dependency graph
5	Feature breakdown	12	Learning order
6	Data flow	13	Top 30 files
7	State management	14	Diagrams

Hakim — Architecture & Codebase Overview

A from-scratch technical map of the Flutter client, built by reading the actual source tree — ~50,800 lines across 90 files. Nothing here is inferred from naming conventions alone; where the codebase doesn't answer a question, that's stated rather than guessed.

1 · High-Level Project Overview

Hakim ("physician" in Arabic) is a bilingual (English/Arabic, full RTL) clinic-management mobile app for small private practices. It is a client for a FastAPI-style REST backend (`backend.hakim-app.cloud`) plus a second, separate AI microservice (`ai-api.hakim-app.cloud`) referenced in config but — as far as this codebase shows — not called directly; voice transcription and imaging analysis go through the main backend, which presumably proxies to the AI service server-side.

Main features

Patient registry · appointment scheduling with types/statuses · a 6-tab **Consultation** workflow per visit (Vitals → Patient History → Voice-to-report AI → AI Medical Imaging → Lab Reports → Manual Report) · AI-assisted reporting (voice→transcript→structured report; image→AI draft→doctor review) · report **finalization** (draft → reviewed → finalized, triggering server-side PDF + WhatsApp delivery) · lab-report PDF upload/viewing · finance/revenue dashboard · admin console (stats, user activation, audit log) · settings (fees, linked assistants, CSV export, language, dark mode) · self-service registration gated behind admin activation.

User roles

Doctor

- Owns a clinic
- Full patient / appointment / consultation / finance access
- Shell: Dashboard, Appointments, Patients, Finance

Assistant

- Linked to exactly one doctor (`doctor_id`)
- Manages that doctor's front-desk workload
- Shell: Dashboard, Appointments, Patients, Payments

Admin

- Cross-clinic oversight
- No clinical data entry
- Shell: Dashboard, Users, Patients, Audit Logs

Overall architecture

A pragmatic **two-layer MVVM** — not a full Clean-Architecture stack. There is **no repository layer, no use-case classes, and almost no domain entity classes**. `ApiService` methods return raw `Map<String,dynamic>` / `List<dynamic>` straight from Dio, and ViewModels store those same raw maps in state. The only real model classes are `UserProfile`, `AppError`, `RegistrationData`, and three small classes living inside

`admin_viewmodel.dart` (`AdminStats` , `AdminUser` , `AuditLog`). Know this before hunting for a `Patient` or `Appointment` class — there isn't one; see §9.

State management is **Riverpod** (`StateNotifierProvider`), theming is **per-role** (three `ThemeExtension` s), navigation is **plain** `Navigator.push` / `MaterialPageRoute` — no named-route table, no router package.

2 · Folder Structure

lib/	
— main.dart	entry point + startup routing
— config/	compile-time constants (base URLs, API key)
— errors/	typed error model + classifier
— services/	two static "backend gateway" classes
— model/	the one shared domain model (UserProfile)
— providers/	Riverpod provider wiring (thin, no logic)
— viewmodels/	StateNotifier business logic, one per role
— views/	screens, grouped by role
— auths/	
— doctor/	
— doctor_pages/	
— consultation/	
— lab_reports/	
— settings/	
— assistant/	
— admin/	
— widgets/	reusable presentational components, by role
— doctor/ · assistant/ · shared/	
— utils/	theming + small pure-function helpers
— splash/	splash screen widget
— l10n/	ARB source + flutter gen-l10n output

FOLDER	WHY IT EXISTS	CONTAINS	TALKS TO
config/	One place to change environment (base URL, key) via <code>--dart-define</code>	app_config.dart	API_Service , auth_viewmodel
errors/	Centralizes "what does this exception mean to a human"	AppError , ErrorHandler	Every ViewModel
services/	Only code allowed to touch Dio / SharedPreferences / secure storage directly	API_Service.dart, settings_service.dart	ViewModels, providers
model/	The one cross-cutting object that survives login	UserProfile.dart	Every role shell / profile screen
providers/	Wires each StateNotifier into Riverpod; deliberately logic-free	One file per ViewModel + theme/locale	Read via <code>ref.watch</code> everywhere
viewmodels/	All business/state logic lives here	5 ViewModels	Calls <code>ApiService</code> ; feeds views
views/.../*_interface.dart	Per-role app shell: top bar, bottom nav, page switcher	—	Hosts role pages; reads role provider
views/doctor/consultation/	The 6 tabs shown inside one active visit	—	doctor_viewmodel , API_Service
widgets/doctor/ · widgets/assistant/	Role-specific presentational components	Cards, forms, avatars, chips	Imported by matching role's views
widgets/shared/	The only two widgets used by more than one role	ErrorRetryWidget , AppDialogs	Imported ad hoc
utils/	Theming + small pure helpers	3 theme extensions, app_themes , digit/clinic helpers	Read via <code>Theme.of(context)</code>
l10n/	Source strings + generated localization classes	app_en.arb, app_ar.arb, generated Dart	AppLocalizations.of(context) everywhere

3 · File Map

Grouped by responsibility. "Depends on" lists significant in-repo imports (framework imports omitted).

CORE

App shell, errors, services, model

FILE	PURPOSE	DEPENDS ON
main.dart	Entry point; inits ApiService, registers global 401 handler, resolves session on cold start, builds MaterialApp	API_Service, auth_viewmodel, settings_service, app_themes, all 3 Interfaces
splash/splash_screen.dart	Branded splash shown while session resolves	—
config/app_config.dart	Base URLs, API key, demo admin creds, page sizes — all <code>--dart-define</code> overridable	—
errors/app_error.dart	<code>AppErrorType</code> enum + <code>AppError</code> exception	—
errors/error_handler.dart	Classifies any exception/DioException into a friendly <code>AppError</code>	app_error.dart
services/API_Service.dart	Entire backend surface — Dio client + interceptor + ~60 endpoint methods	app_config, error_handler
services/settings_service.dart	Local device prefs: fees, language, dark mode, notifications, last-active-tab per role	—
model/UserProfile.dart	Cross-cutting user identity + role + clinic fields + assistant→doctor linkage	—

Providers

LOGIC-FREE WIRING

FILE	PROVIDER(S)	NOTES
auth_providers.dart	<code>authProvider</code> , <code>authLogoutCleanupProvider</code>	Cleanup provider invalidates the 3 role providers on logout
doctor_providers.dart	<code>doctorViewModelProvider</code>	No <code>autoDispose</code> — keeps cached visit/report IDs across navigation
assistant_providers.dart	<code>assistantViewModelProvider</code>	No <code>autoDispose</code> — keeps resolved linked-doctor ID
admin_providers.dart	<code>adminViewModelProvider</code>	—
settings_providers.dart	<code>settingsViewModelProvider</code>	—
theme_providers.dart	<code>themeModeProvider</code> , <code>scaffoldMessengerKey</code>	Key lets snackbars render above modal sheets
locale_provider.dart	<code>localeProvider</code>	Restores saved language on construction

ViewModels

FILE	STATE	PURPOSE
CORE auth_viewmodel.dart	AuthState	Login (JWT decode, role resolution, discovery fallback), self-registration (raw http , not Dio), logout
DOCTOR doctor_viewmodel.dart 1,626 ln	DoctorState	Patients/appointments/vitals CRUD+pagination, visit lifecycle, voice/report assembly, image upload, finalization
ASSISTANT assistant_viewmodel.dart 1,065 ln	AssistantState	Mirrors DoctorViewModel, scoped to one linked doctor; doctor-resolution chain in <code>initForAssistant()</code>
ADMIN admin_viewmodel.dart	AdminState	Dashboard stats (+ client aggregation fallback), combined user list, audit log pagination
CORE settings_viewmodel.dart	SettingsState	Prefs, password change, CSV export, fee-sync-to-backend with 409 recovery

AUTH

views

views/auths/login_page.dart	Single login for all 3 roles; force-pins English/LTR regardless of app locale; routes via <code>AuthState.destination</code>
views/auths/registration_page.dart	Tabbed Doctor/Assistant self-registration; "pending approval" screen on success

DOCTOR

pages, consultation, lab reports, settings

doctor_interface.dart	Shell: top bar, 4-tab bottom nav, page switcher, logout
doctor_dashboard.dart	Overview cards, "up next", revenue-today, fl_chart trend
doctor_appointments_page.dart	List/calendar, filters, launches consultation
doctor_patients_page.dart	Patient search/list/create
doctor_patient_detail.dart 1,820 ln	Full record: info, conditions, appointments, reports, images, labs
doctor_finance_page.dart	Revenue table/summary
doctor_profile_page.dart / doctor_edit_profile_page.dart	View/edit own profile
doctor_consultation_page.dart 1,095 ln	Consultation container. Resolves/creates the visit, hosts 6-tab TabBarView, owns finalize-on-complete orchestration
doctor_settings_page.dart 1,761 ln	Settings entry sheet; hosts sub-sheets
consultation/vitals_tab.dart	BP/pulse/temp/weight + condition tagging
consultation/patient_history_tab.dart 2,092 ln — largest view file	7 collapsible sections aggregating full patient history (allergies section has no backend yet)
consultation/voice_tab.dart + widgets/doctor/voice_recording_widget.dart	Record → upload → transcribe
consultation/voice_report_review_page.dart	Editable AI-generated report draft
consultation/ai_imaging_tab.dart + ai_imaging_review_page.dart 1,701 ln	Pick/capture → upload → AI draft (priority chain) → edit → staged until completion
consultation/manual_report_tab.dart	Free-form report, no AI
lab_reports/lab_reports_page.dart	<code>LabReportsPage</code> (standalone) + <code>LabReportsTab</code> (embedded)
lab_reports/lab_report_viewer_page.dart	Authenticated download then open locally via <code>open_filex</code>
settings/*.dart	appearance, change-password, default-fees, linked-assistants, export sheets

ASSISTANT views

assistant_interface.dart	Shell; wraps subtree in Assistant Theme explicitly (dialogs re-wrap manually)
assistant_dashboard.dart	Today's appointments, quick stats for linked doctor
assistant_appointments_page.dart	Booking, check-in, payment toggle
assistant_patients_page.dart	List/search/create — delegates detail to widgets/assistant
assistant_vitals_page.dart	Pre-consultation vitals entry
assistant_payments_page.dart	Paid/unpaid ledger
assistant_settings_page.dart 1,885 ln	Settings + linked-doctor info + export, all as private classes in one file
views/assistant/assistant_patient_detail.dart	⚠ Dead file — same lineage as <code>widgets/assistant/assistant_patient_detail.dart</code> , but nothing imports this copy. Safe to delete; out of scope here.

ADMIN views

admin_interface.dart	Shell; indigo AdminTheme
admin_dashboard.dart	6-card stats grid, skeletons, retry-on-error
admin_users_page.dart	Tabbed doctor/assistant/admin list, search, filters, activate/deactivate
admin_patients_page.dart	Read/moderate patients system-wide
admin_audit_logs_page.dart 975 ln	Paginated, filterable audit trail
admin_settings_page.dart 1,116 ln	Profile (read-only), password, appearance, sign out — no fee/assistant sections

Widgets & Utils

DOCTOR widgets	doctor_shared_widgets , doctor_pat_card , doctor_appt_card , doctor_appt_form , doctor_pat_form , doctor_appt_details_sheet , doctor_consultation_widgets , image_upload_widget , ai_analysis_result_widget , voice_recording_widget
ASSISTANT widgets	assistant_shared_widgets , assistant_pat_card , assistant_appt_card , assistant_pat_form , assistant_appt_form , assistant_patient_detail
SHARED widgets	error_retry_widget , app_dialogs — only two widgets used by more than one role
utils	doctor_theme / assistant_theme / admin_theme (ThemeExtension + brand tokens), app_themes.dart 1,293 ln (assembles both global ThemeData), arabic_digits.dart , clinic_helpers.dart
l10n	app_en.arb / app_ar.arb source strings + generated Dart (do not hand-edit)

4 · Entry Point

main.dart → `main()` :

1

ApiService.init()

Constructs the single Dio instance — base URL, timeouts, the X-API-KEY + Bearer request interceptor, response/error logging, global 401 handling.

2

registerUnauthorizedHandler(...)

Wires a callback that resets AuthViewModel's static instance and force-navigates to LoginPage on any 401, anywhere.

3

runApp(ProviderScope(child: MyApp()))

MyApp watches themeModeProvider + localeProvider; home is `_StartupRouter`.

4

_StartupRouter shows SplashScreen

Then calls `_resolveInitialScreen()` in initState.

5

Read token / role / profile

From secure storage. Missing or JWT `exp` already passed (decoded client-side, no crypto lib) → LoginPage.

6

Rebuild UserProfile from cache

Including doctorId/doctorEmail for assistants — skips the expensive discovery lookup on warm restarts.

7

Read last-active tab

From SettingsService — default tab 1 (Appointments) for assistants, tab 0 otherwise.

8

pushReplacement to the role shell

Pre-seeded with profile + tab index; 300ms fade. Splash is never seen again this process lifetime.

5 • Feature Breakdown

FEATURE	MAIN SCREENS	VIEWMODEL	APIS
CORE Authentication	LoginPage, RegistrationPage	AuthViewModel	POST /auth/login , /users/{doctors,assistants}/register (raw http), /auth/logout , /auth/change-password
DOCTOR Dashboard	DoctorDashboardPage	DoctorViewModel	Derived from already-loaded state; fl_chart
Patients	Role-specific list/detail pages	Doctor/Assistant/AdminViewModel	/clinic/patients , /clinic/conditions
Appointments	Role variants	Doctor/AssistantViewModel	/clinic/appointments , /clinic/appointment-types
DOCTOR Consultation / Visits	DoctorConsultationPage + 6 tabs	DoctorViewModel	/clinic/visits , /clinic/patient-vitals
AI Voice Reporting	VoiceTab → VoiceReportReviewPage	DoctorViewModel	record_pkq → POST /reports/transcribe → /reports/medical-reports
AI Medical Imaging	AllImagingTab → AllImagingReviewPage	DoctorViewModel	image_picker → POST /reports/medical-images → PATCH .../review (deferred)
Manual Reports	ManualReportTab	DoctorViewModel	POST /reports/medical-reports , PATCH .../status
Report Finalization	DoctorConsultationPage._save	DoctorViewModel.finalizeReport	POST .../finalize (120s timeout, sync PDF+WhatsApp)
Lab Reports	LabReportsPage/Tab, LabReportViewerPage	DoctorViewModel	POST /reports/lab-reports , authenticated download
Patient History (in-consultation)	PatientHistoryTab	DoctorViewModel	Parallel fetch: visits + reports + images + per-visit labs
Finance	DoctorFinancePage, AssistantPaymentsPage	Doctor/AssistantViewModel	Derived from appointment fee / is_paid — no dedicated endpoint
ADMIN Console	Dashboard, Users, Patients, Audit Logs	AdminViewModel	/admin/dashboard/stats , /users/* , /reports/audit-logs
Settings	Bottom-sheet stacks per role	SettingsViewModel (doctor) / inline (others)	SettingsService , changePassword , fee sync
Localization / Theming	—	LocaleNotifier, themeModeProvider	shared_preferences

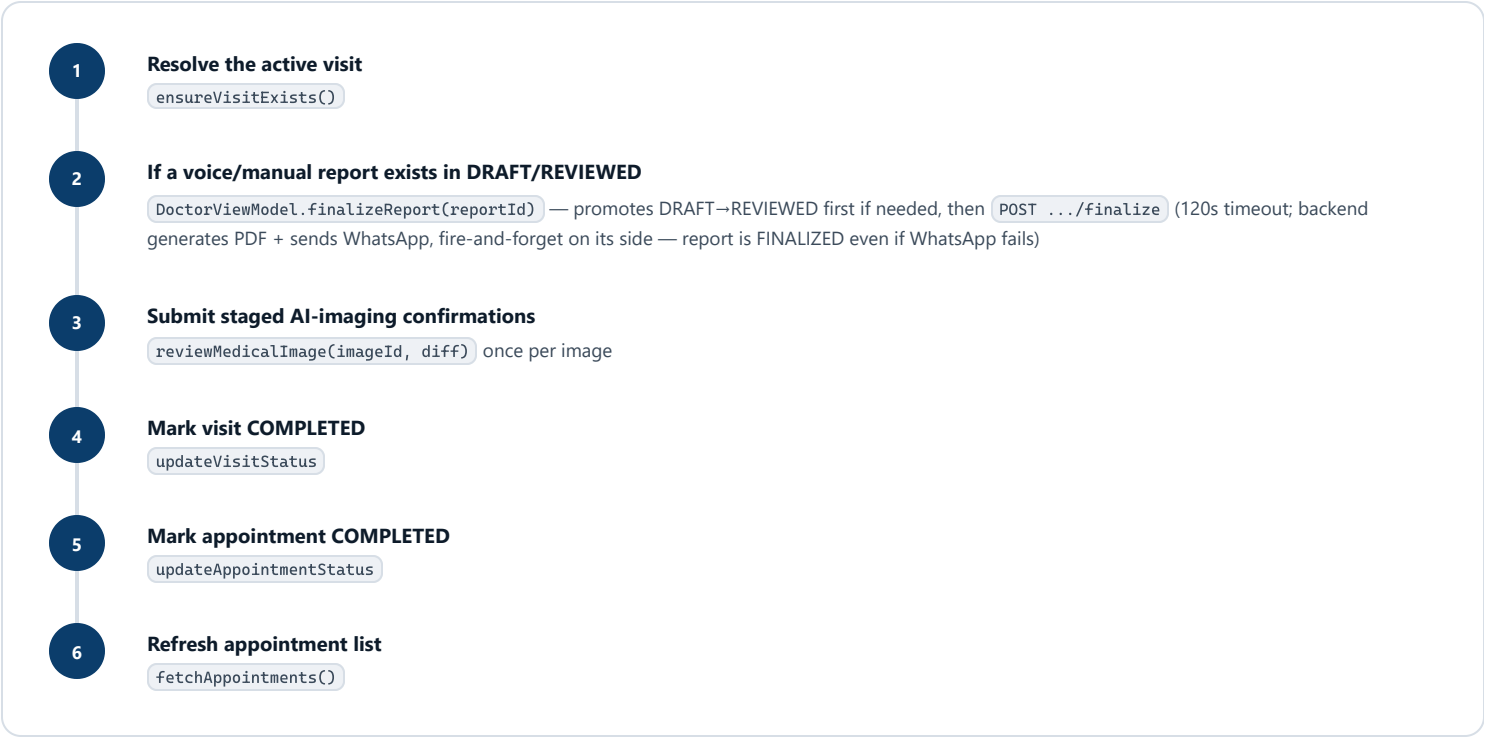
6 · Data Flow

Standard direction — e.g. loading the patient list:



No manual `setState` round-trip is needed — Riverpod's `ref.watch` triggers a rebuild whenever `copyWith` produces a new state object.

The one workflow worth tracing end-to-end: consultation completion



Imaging-only consultations (no voice/manual report) skip step 2 entirely — there is no "anchor" empty report created for them; a follow-up fix removed that behavior because it caused a duplicate WhatsApp send.

7 · State Management

- **Solution:** `flutter_riverpod ^2.5.1` — `StateNotifierProvider` + `StateNotifier<T>` for every stateful feature, plus a couple of plain `StateProvider` s and one hand-rolled notifier for locale.
- **Initialization:** a single `ProviderScope` wraps the app in `main()` . No provider overrides anywhere (no test/mock wiring present).
- **Managing classes:** `AuthViewModel` , `DoctorViewModel` , `AssistantViewModel` , `AdminViewModel` , `SettingsViewModel` , `LocaleNotifier` — each paired with an immutable `*State` + `copyWith` .
- **UI updates:** views are `ConsumerWidget` / `ConsumerStatefulWidget` ; `ref.watch` rebuilds, `ref.read(...).method()` triggers actions, `ref.listen` runs one-shot side effects like post-login navigation.
- **No `autoDispose`** : called out explicitly in three of the four provider files' own comments — state survives navigation so cached IDs/lookups aren't repeated, at the cost of needing manual invalidation on logout (`authLogoutCleanupProvider`).
- **Local-only UI state** (tab index, form controllers, animations) stays as plain `State` / `setState` — not lifted into Riverpod.

8 · API Layer

- **Single gateway:** `lib/services/API_Service.dart` — one static `Dio` instance, ~60 endpoint methods. Exception: `auth_viewmodel.dart` uses the raw `http` package (not `_dio`) for the two self-registration endpoints — those calls bypass the shared interceptor entirely, so they get no standardized error-detail extraction and no 401 handling.
- **Dio config:** base URL from `AppConfig.backendBaseUrl` (default `https://backend.hakim-app.cloud`), 30s connect / 60s receive timeout (bumped to 120s just for `finalizeReport`).
- **Interceptor:** `onRequest` injects `X-API-KEY` + Bearer token and logs the call; `onResponse` logs status; `onError` extracts a readable `detail` from FastAPI-style bodies and — on **401** — clears stored credentials and fires the app-wide logout callback, guarded against double-firing.
- **Base URL / key location:** `lib/config/app_config.dart` — all four values are `String.fromEnvironment` with hardcoded fallbacks, meaning the production API key currently ships baked into the client binary as a fallback constant.
- `aiApiBaseUrl`: defined but not referenced anywhere else in `lib/` — voice/imaging calls go through the main backend. Worth confirming with the backend team whether a direct-to-AI path is planned.
- **Auth flow:** `POST /auth/login` → JWT stored in `flutter_secure_storage` → role decoded client-side from the payload (base64url-decoded manually, **not signature-verified** — acceptable here since claims only drive UI routing, never authorization) → profile enriched by cross-referencing `/users/doctors` / `/users/assistants` by email → persisted for session restore.
- **Token storage:** `flutter_secure_storage`, keys `access_token` / `user_role` / `user_profile` (a JSON blob of the whole profile, so cold start redraws the UI with zero network calls).
- **Error handling:** `errors/error_handler.dart` classifies everything into an `AppError` with a log-only `technical` string and a display-ready `userMessage`.

9 · Models

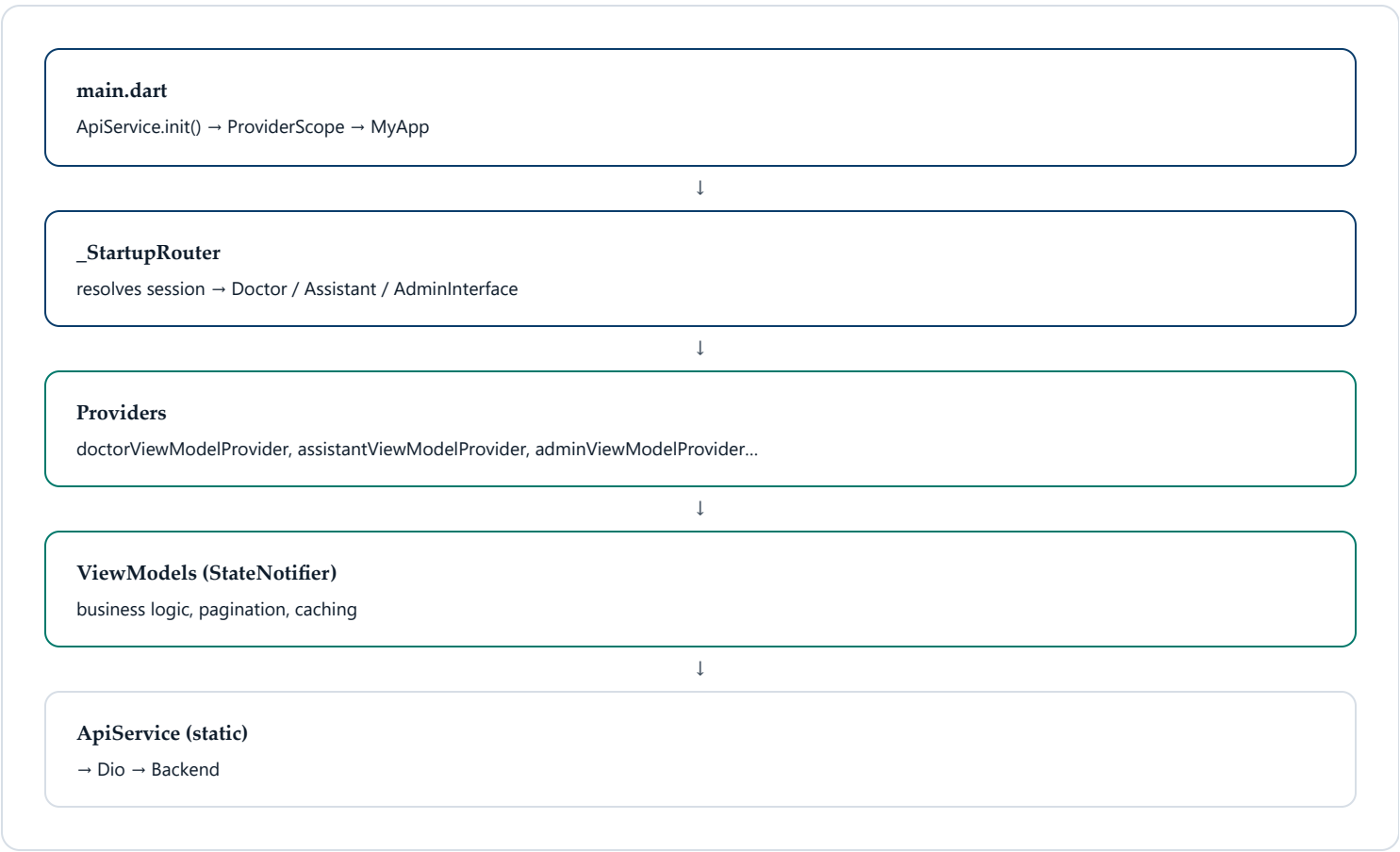
MODEL	REPRESENTS	BUILT FROM / USED BY
UserProfile	The logged-in user across all 3 roles, incl. assistant→doctor linkage	AuthViewModel._buildProfile; restored from secure storage cold-start
RegistrationData	Self-registration form payload	RegistrationPage → AuthViewModel.register()
AppError / AppErrorType	A classified, typed exception with a user-safe message	ErrorHandler.parse()
AdminStats	Aggregate counts for the admin dashboard	/admin/dashboard/stats or client aggregation
AdminUser	Normalized user row; distinguishes profile-table PK from users-table PK for status updates	/users/{doctors,assistants,admins}
AuditLog	One audit entry, tolerant of several backend field-name variants	/reports/audit-logs

Everything else — Patient, Appointment, Visit, MedicalReport, MedicalImage, LabReport, Vitals, AppointmentType — is a raw `Map<String,dynamic>`, never a typed class. This is consistent across every ViewModel and widget (e.g. `appointment['patient_first_name']`). It's the single biggest thing to internalize before changing this code: there's no compiler-enforced schema for these objects, and field-name mismatches with the backend (several fixed in code comments — e.g. lab-report `report_url` vs `report_file`) are a recurring bug source precisely because of this.

10 · Navigation

- **Routing system:** none of the router packages — plain `Navigator.push(MaterialPageRoute(...))` for drill-in, `pushReplacement` / `pushAndRemoveUntil` for full-stack swaps.
- **Initial route:** decided imperatively in `main.dart`'s `_resolveInitialScreen()` — a widget returned by a function, not a named route.
- **Named routes:** not used — `MaterialApp` has no `routes:` / `onGenerateRoute:`.
- **Role-based navigation:** each role shell owns a hand-built bottom nav (a `Row` of `InkWell`s, not Flutter's `BottomNavigationBar`). Selected index is persisted per-role via `SettingsService.setLastTab`. Switching roles only happens by logging out and back in.
- **Consultation sub-nav:** a `TabController`-driven `TabBarView`, 6 tabs, `NeverScrollableScrollPhysics` — reachable only via the `TabBar`, not swipe.

11 · Dependency Graph



In parallel: **Views** read Providers directly and render **Widgets**, styled through the three **Theme extensions** aggregated by `AppThemes.light` / `.dark`. Cross-cutting, read from almost every layer: `l10n/generated/app_localizations.dart` and `errors/*`.

12 · Learning Order

Fastest path to a working mental model of this specific codebase:

1

main.dart

Startup + session resolution

2

config/app_config.dart + services/API_Service.dart

What the backend looks like from the client's point of view

3

errors/app_error.dart + error_handler.dart

How failures surface everywhere else

4

model/UserProfile.dart

The one shared identity object

5

auth_viewmodel.dart + login_page.dart

Login → role resolution → routing

6

doctor_providers.dart → doctor_viewmodel.dart

`DoctorState` first, then CRUD — the template every other ViewModel follows

7

doctor_interface.dart

How a role shell wires provider + tabs + theme together

8

doctor_patients_page.dart / doctor_appointments_page.dart

The standard list/search/paginate/create pattern, reused everywhere

9

doctor_consultation_page.dart

Most complex single screen — read after step 6

10

consultation/* in tab order

vitals → history → voice → imaging → lab → manual

11

assistant_viewmodel.dart

Mostly a mirror of step 6 — focus on the doctor-resolution chain

12

admin_viewmodel.dart + views/admin/*

Smallest, most self-contained role

13

settings_viewmodel.dart + doctor/settings/*

Cross-cutting preferences, briefly touches every backend area

14

app_themes.dart + one role theme

Theming mechanics

15

l10n/app_en.arb

Skim for the localization key vocabulary — skip the generated Dart

13 · Files Ranked by Importance

Ranked for "read this to actually understand the app," not by line count alone.

#	FILE	REASON	DIFFICULTY
1	services/API_Service.dart	Defines the entire backend contract the app is built against	Medium
2	main.dart	Entry point, session bootstrap, global 401 wiring	Low
3	viewmodels/doctor_viewmodel.dart	Largest, most representative business-logic file — the template for the other three	High
4	viewmodels/auth_viewmodel.dart	Login/registration/role-resolution, incl. JWT-decode-and-fallback-scan	Medium
5	doctor_pages/doctor_consultation_page.dart	Core clinical workflow — visits, 6 tabs, finalization	High
6	model/UserProfile.dart	Small but load-bearing — every "who's logged in" screen depends on it	Low
7	errors/error_handler.dart	Explains why error messages look the way they do everywhere	Low
8	errors/app_error.dart	The typed-error vocabulary used in every catch block	Low
9	viewmodels/assistant_viewmodel.dart	2nd-largest ViewModel; doctor-linkage resolution is the one truly new idea	High
10	config/app_config.dart	Tiny, but explains every base URL/key you'll see logged	Low
11	consultation/patient_history_tab.dart	Largest view file — the "aggregate several endpoints into one screen" pattern	High
12	doctor_pages/doctor_interface.dart	Canonical role shell: top bar + tabs + theme + logout	Low
13	viewmodels/admin_viewmodel.dart	Compact; introduces the profile-PK vs users-table-PK distinction	Medium
14	views/auths/login_page.dart	<code>ref.listen</code> -for-navigation pattern + forced-English-locale override	Medium
15	providers/auth_providers.dart	Small, but explains cross-provider logout-invalidation	Low
16	consultation/ai_imaging_review_page.dart	Most intricate single interaction: stage-locally-then-submit-once-at-finalize	High
17	viewmodels/settings_viewmodel.dart	The create-then-recover-by-fetch pattern for fee-default sync	Medium
18	lab_reports/lab_report_viewer_page.dart	Authenticated-download-then-open-locally — needed for any backend-hosted file	Medium
19	utils/app_themes.dart	How 3 role themes coexist inside 2 global ThemeDatas	Medium
20	doctor_pages/doctor_dashboard.dart	Composing already-loaded state into charts/cards	Medium
21	views/assistant/assistant_interface.dart	The "re-wrap dialogs in Theme manually" gotcha	Low
22	views/admin/admin_users_page.dart	Standard filtered/searchable list pattern, admin variant	Medium
23	consultation/voice_report_review_page.dart	AI-draft-review UX pattern (mirrored by imaging review)	Medium
24	consultation/manual_report_tab.dart	Simplest report-entry tab — good baseline before voice/imaging	Low
25	doctor_pages/doctor_patient_detail.dart	Largest patient-facing screen — aggregates conditions/appointments/reports/images/labs	High
26	providers/assistant_providers.dart	One line of wiring, but its comment explains why no autoDispose — read once, apply everywhere	Low
27	views/admin/admin_audit_logs_page.dart	Pagination + multi-filter UI pattern at its most developed	Medium
28	utils/clinic_helpers.dart	Small helper shared by both clinical ViewModels — know it exists before duplicating logic	Low
29	doctor_pages/doctor_settings_page.dart	Largest settings file — bottom-sheet-of-bottom-sheets composition	High
30	l10n/app_en.arb	Not Dart, but the fastest way to see every user-facing string key	Low

14 · Diagrams

Overall architecture

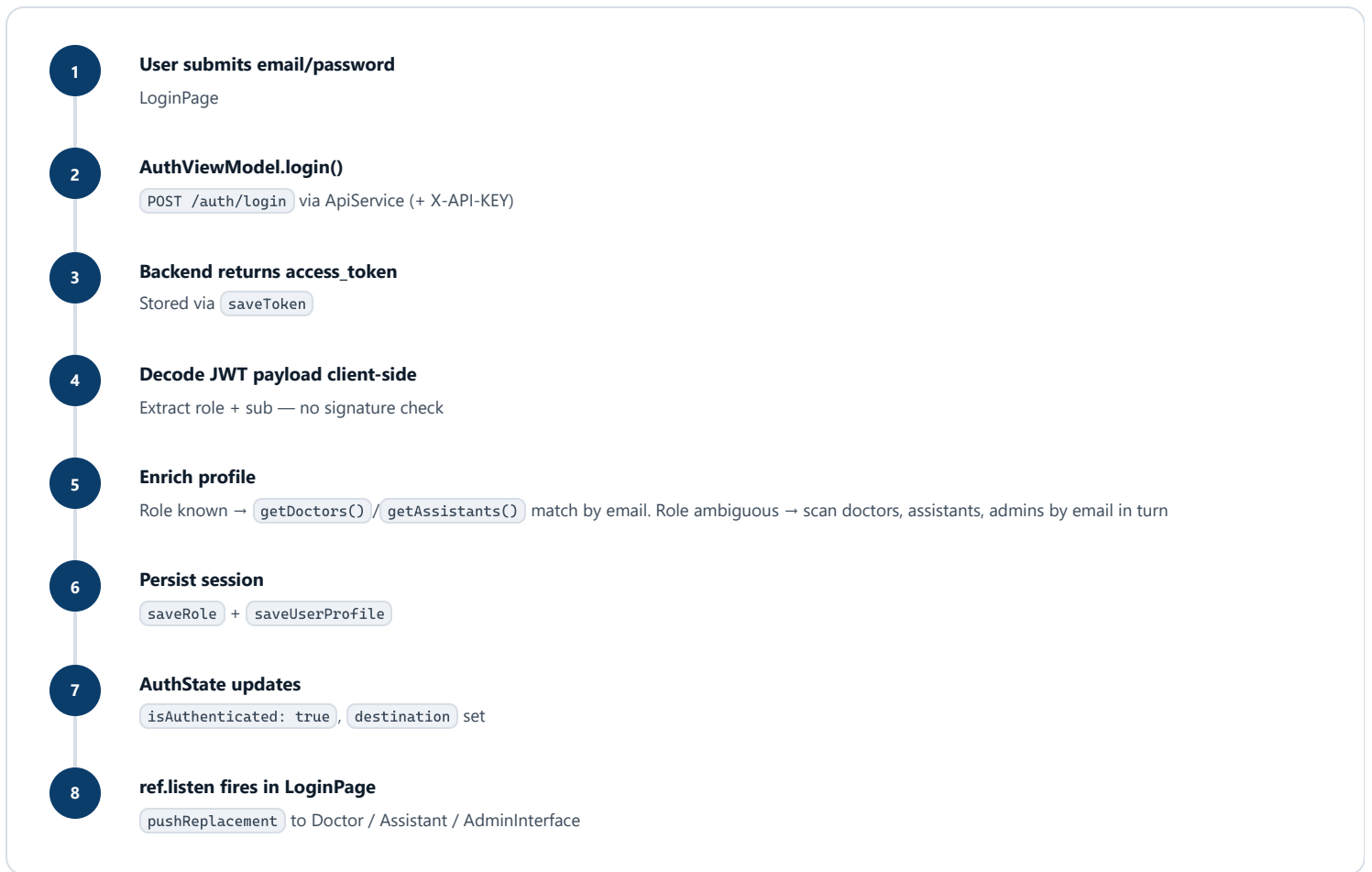


A 401 anywhere in the Gateway layer short-circuits straight up to the global logout handler registered in `main.dart`, bypassing the normal state flow.

Folder relationships



Login flow



Navigation flow



Notes on report scope. Per the task rules, this document is analysis-only — no code was modified, refactored, or generated. Two things surfaced during the read that a maintainer may want to act on separately:

- `views/assistant/assistant_patient_detail.dart` appears to be an unused duplicate of `widgets/assistant/assistant_patient_detail.dart` — only the `widgets/` copy is imported anywhere.
- `AppConfig.aiBaseUrl` is defined but never referenced by any other file in `lib/` — worth confirming whether a direct-to-AI client path is planned.